RNAseq-AMD Project Handoff Guide

Autoencoder / clustering codebases (CNC-VAE, CNC-DEC, DEC, X-DEC)
and the MEGENA_AMI gene co-expression network analysis (R)

July 20, 2026

Written for the student inheriting this project. Each codebase also has its own README.md in its project folder — this document collects all of them in one place, plus an overview of how the pieces fit together.

Table of Contents

1. Project Overview

This project studies age-related macular degeneration (AMD) severity (the 'mgs_level' phenotype, e.g. MGS1 = mild vs. MGS4 = severe) using two independent analysis tracks that both start from the same underlying cohort:

- **Shared input data:** RNA-seq CPM gene expression (Dataset/aak100_cpmdat.csv) and clinical/genotype metadata (Dataset/MetaSheet_1_4.csv), matched by sample_id.

The two tracks:

- **Autoencoder / clustering track (Python):** Four PyTorch codebases that learn a compressed "latent embedding" of each patient (clinical + gene expression combined) and/or cluster patients directly from that embedding. Sections 2–5 below.
- **Gene network track (R):** An R pipeline (MEGENA) that instead builds a gene-gene co-expression network and looks for groups of genes ("modules") whose combined expression differs between AMD severity groups. Section 6 below.

These two tracks answer different questions — "can we compress a patient into a handful of numbers that separate AMD severity" vs. "which groups of genes move together, and does that group's behavior track AMD severity" — and don't depend on each other. You can work on either independently.

1.1 The four Python codebases, at a glance

Folder	Model	Produces
CNC-VAE	Single-input VAE (clinical+RNA concatenated)	A latent embedding only — no clustering.
CNC-DEC	CNC-VAE + Deep Embedded Clustering	A latent embedding AND cluster assignments.
DEC	Plain (non-variational) MLP autoencoder + DEC	Same as CNC-DEC, simplest architecture, no VAE regularization.
X-DEC	Dual-branch VAE (numeric and binary kept separate) + DEC	Same as CNC-DEC, but clinical and RNA features are never concatenated — each gets its own encoder branch.

All four share the same core ideas: a 'latent size' hyperparameter (how many numbers each patient gets compressed into — try smaller values like 4 or 8 to see if clusters/separation improve or degrade), and (for the three DEC variants) a two-stage train-then-cluster pipeline. Read CNC-VAE's section first even if you only need X-DEC or DEC — the later sections assume you've seen the basic VAE architecture and don't re-explain it.

1.2 Before you start

- Python 3.9+ with pip. Each folder has its own requirements.txt (pip install -r requirements.txt from inside that folder).
- R (any recent version) with RStudio, for the MEGENA_AMI scripts (Section 6) — completely separate from the Python setup above.
- All four Python projects expect Dataset/aak100_cpmdat.csv and Dataset/MetaSheet_1_4.csv to exist two directories up from the project folder (i.e. a sibling Dataset/ folder at the RNAseq-AMD repo root) — this is handled automatically by each project's misc/dataset.py, you don't need to pass a path unless you want to override it.

2. CNC-VAE — Clin+mRNA Integration

A PyTorch port of the CancerAI-CL CNC-VAE ("Concatenated iNputs VAE"): a single-input variational autoencoder that concatenates clinical/genotype features with gene-expression features into one vector, compresses it down to a small latent embedding, and reconstructs the input from that embedding. Ported to PyTorch (rather than the original TensorFlow/Keras) to avoid TensorFlow's DLL-loading problems on Windows.

2.1 How it works

```
[clinical features | gene expression features]  (concatenated into one vector)
        |
        v
  Linear -> BatchNorm -> activation        (encoder hidden layer, width = --ds)
        |
   +----+----+
   v         v
 fc_mu   fc_logvar     (each Linear(ds -> ls) - the latent mean/log-variance heads)
   |         |
   +----+----+
        v
  reparameterization trick -> z   (the latent embedding, dimension = --ls)
        |
        v
  Linear -> BatchNorm -> activation -> Dropout -> Linear   (decoder, mirrors the encoder)
        |
        v
  reconstructed [clinical | gene expression] vector
```

Loss = MSE reconstruction error + beta * distance, where distance is either KL divergence to a standard normal prior (--distance kl) or Maximum Mean Discrepancy (--distance mmd). beta controls how much the latent space is pulled toward the prior vs. how accurately the input is reconstructed (this is the "beta-VAE" idea). Core model code: models/cncvae.py — _CNCVAENet is the raw PyTorch network, CNCVAE is a wrapper that owns training/prediction.

2.2 The 4 scripts in this folder

Script	Purpose
run_cncvae.py	Command-line training: whole-cohort or stratified-CV-fold mode. Saves the latent embedding as .npz.
compare_representations.py	Reproduces the CNC_VAE_AMD_V3 reference notebook: auto-tunes beta, compares CNC-VAE latent vs. PCA vs. raw features with 5-fold CV classifiers, plots t-SNE, saves the embedding as .csv.
analyse_representations.py	Reads the .npz files run_cncvae.py wrote and scores simple classifiers (logreg/nb/svm/rf) on them, optionally with a t-SNE plot. Does not retrain anything.
load_encoder.py	Reloads a saved encoder checkpoint (encoder_cncvae*.pt) and re-encodes data into its latent space without retraining.

2.3 What can be changed

Everything below is a command-line flag — no code editing required. The latent size is the main thing worth experimenting with:

```
py run_cncvae.py --ds 64 --ls 8    # try 8-dimensional latent space
py run_cncvae.py --ds 64 --ls 4    # try 4-dimensional latent space
```

--ls sets the width of the bottleneck (fc_mu/fc_logvar in models/cncvae.py). run_cncvae.py's output directory name already encodes --ls/--ds/--distance/--beta so different runs don't overwrite each other. compare_representations.py does NOT do this automatically — give each run its own --out folder if you don't want to overwrite the previous run's comparison_table.csv / cncvae_latent_embedding.csv.

Flag	Meaning
--ds	Hidden/dense layer width. run_cncvae.py only accepts 16/32/64/128 (each preset also fixes epochs/batch-size/dropout/activation); compare_representations.py takes any int with separate flags for those.
--beta	Beta-VAE regularization weight (or auto-resolve via --target_ratio in compare_representations.py).
--distance	kl or mmd.
--clinical_mode	'full' (every clinical column) or 'curated' (age/sex/genotype + prevalence-filtered, leakage-excluded history flags).
--fold	(run_cncvae.py only) 0 for whole-cohort training, or 1..--numfolds for one stratified CV fold.
--label_col	Phenotype column for CV stratification / reporting (default mgs_level).
--save_model	Also save the encoder as a standalone, reloadable .pt checkpoint.

2.4 Packages needed

```
pip install -r requirements.txt
```

Installs: torch>=2.0, numpy>=1.24, pandas>=1.5, scikit-learn>=1.2, matplotlib>=3.7, shap>=0.44.

2.5 How to run

```
# Train on the whole cohort, latent size 16, hidden width 64
py run_cncvae.py --ds 64 --ls 16

# Train + evaluate one stratified CV fold (fold 1 of 5)
py run_cncvae.py --ds 64 --ls 16 --fold 1

# Reproduce the reference-notebook comparison table + t-SNE + embedding CSV
py compare_representations.py --ds 256 --ls 16 --epochs 150 --bs 16 --out comparison

# Score the embeddings run_cncvae.py --fold 0 produced
py analyse_representations.py --resdir
results/CNCVAE_Clin+mRNA_integration/cncvae_LS_16_DS_64_kl_beta_1.0 --numfolds 0

# Reload a saved encoder and re-encode without retraining
py load_encoder.py --encoder comparison/encoder_cncvae.pt --clinical_mode curated --
normalization log1p_minmax
```

2.6 Outputs

- run_cncvae.py -> results/CNCVAE_Clin+mRNA_integration/cncvae_LS_{ls}_DS_{ds}_{distance}_beta_{beta}/ containing {label_col}.npz (or {label_col}{fold}.npz per fold), {label_col}_labels.npz, and (with --save_model) encoder_cncvae.pt / vae_cncvae.pt.

- compare_representations.py -> --out folder containing comparison_table.csv, tsne_comparison.png, cncvae_latent_embedding.csv, and (with --save_model) encoder_cncvae.pt.
- analyse_representations.py -> --out folder (default analyses/) containing a results CSV and optionally a t-SNE PNG.

2.7 Reloading a saved encoder — an important caveat

The latent size CANNOT be changed on a saved model — it's baked into the weight matrix shapes at training time (fc_mu/fc_logvar: ds->ls, decoder's first layer: ls->ds). To try a different --ls, retrain from scratch; there's no shortcut via a checkpoint. What a saved checkpoint IS good for is re-encoding data without retraining, using the exact architecture it was saved with — that's what load_encoder.py does. You must tell it which preprocessing the checkpoint was originally trained with via --normalization ('minmax' for run_cncvae.py, 'log1p_minmax' for compare_representations.py). Getting this wrong won't error — it will just silently produce meaningless embeddings — so double check which script produced the checkpoint you're loading.

3. CNC-DEC — CNC-VAE + Deep Embedded Clustering

Combines the CNC-VAE encoder (Section 2) with Deep Embedded Clustering (DEC): pretrains the VAE on reconstruction, then fine-tunes the encoder jointly with a clustering head so the latent space itself becomes more clusterable. Produces --n_clusters clusters directly from the data — no label is used for training, only (optionally) for reporting how well the discovered clusters line up with a known phenotype afterward. This is the single-concatenated-input variant (clinical + gene expression concatenated into one vector, same as CNC-VAE). For the dual-branch variant, see Section 5 (X-DEC).

3.1 How it works

Two stages, both run automatically by one call to run_cncdec():

- **1. Pretraining —** An ordinary CNC-VAE (see Section 2's architecture diagram) is trained on reconstruction only, giving the encoder a reasonable starting latent space.
- **2. DEC self-training —** K-means initializes cluster centers in that latent space; then, in a loop, the model computes soft cluster assignments Q (via a Student's-t kernel, same kernel t-SNE uses), sharpens that into a target distribution P, and takes a gradient step pulling Q toward P. Repeated until few samples change cluster assignment between updates (--dec_tol) or --dec_maxiter is reached. Critically, the encoder's weights are updated too — not just the cluster centers — so the latent space itself is reshaped to be more clusterable.

Model code: models/cncvae.py (same architecture as CNC-VAE's, just built from constructor kwargs instead of an argparse.Namespace, so it's usable standalone); models/dec.py (the DECHead clustering layer and the DEC class that runs the self-training loop); models/metrics.py (calculate_metrics(): clustering accuracy via Hungarian matching, NMI, ARI — reporting only, never used for training).

3.2 How to run

```
py run_cncdec.py --ds 64 --ls 16 --n_clusters 2
```

Or work interactively in CNC-DEC Runner.ipynb, which trains, saves results to results/manual_run/, then plots: (1) a 2-panel PCA scatter of the latent space (predicted clusters vs. true mgs_level), (2) a cluster-vs-label crosstab heatmap (the confusion-matrix-style view), (3) a histogram of cluster-assignment confidence.

3.3 What can be changed

Flag	Meaning
--ls	Latent dimension size (default 16) — the main knob to experiment with.
--ds	Encoder/decoder hidden layer width (default 64).
--n_clusters	Number of clusters DEC should find (default 2) — set to match how many phenotype groups you're comparing against.
--distance / --beta	VAE pretraining regularizer (kl or mmd) and its weight.
--epochs / --bs	CNC-VAE pretraining epochs / batch size.
--dec_maxiter / --dec_update_interval / --dec_batch_size / --dec_tol	DEC self-training loop controls.
--n_init	Number of K-means restarts used to initialize cluster centers.
--clinical_mode	'curated' (default, recommended for this cohort size) or 'full'.
--label_col	Phenotype column reported against discovered clusters (never used in training). Default mgs_level.
--save_model	Save the trained encoder as encoder_cncvae.pt (same checkpoint

	format as CNC-VAE).
--stability	Also run a repeated K-fold cluster-stability check (retrains k*rep times — slow).

Output directory name already encodes --ls/--ds/--distance/--beta/--n_clusters: results/CNCDEC_Clin+mRNA_integration/cncdec_LS_{ls}_DS_{ds}_{distance}_beta_{beta}_k{n_clusters}/.

3.4 Packages needed

```
pip install -r requirements.txt
```

Installs: torch>=2.0, numpy>=1.24, pandas>=1.5, scikit-learn>=1.2, scipy>=1.10 (Hungarian-matching accuracy metric), matplotlib>=3.7 (needed for CNC-DEC Runner.ipynb's plots).

3.5 Outputs

run_cncdec.py writes to results/CNCDEC_Clin+mRNA_integration/cncdec_LS_..._k{n}/:

- {label_col}.npz — latent embedding z, y_pred (cluster labels), y_proba (soft assignments), centroids, plus y/sample_id/classes for interpretation.
- cncdec_latent_embedding.csv — same embedding in the CSV shape the XGBoost pipeline expects.
- encoder_cncvae.pt (with --save_model); cluster_stability.csv (with --stability).

Console output reports acc/ari/nmi against --label_col both periodically during DEC training and at the end.

4. DEC — Plain Deep Embedded Clustering

The simplest of the four variants: a plain (non-variational) MLP autoencoder combined with Deep Embedded Clustering. No VAE regularization, no dual-branch input handling — just reconstruction + clustering. Port of the DAM-IC Deep-Embedded-Clustering-generalisability-and-adaptation-for-mixed-data-types repo's plain single-input path, rewritten in PyTorch. If you want the VAE-regularized version, see Section 3 (CNC-DEC, single concatenated input) or Section 5 (X-DEC, dual-branch).

4.1 How it works

```
[gene expression (min-max scaled) | clinical dummies]  (concatenated into one vector)
        |
        v
  Linear -> ReLU              (hidden layer, width = --neurons_h)
        |
        v
  Linear -> ReLU              (bottleneck/latent layer, width = --neurons_e
                               -- THIS is the embedding DEC clusters on)
        |
        v
  Linear -> ReLU -> Linear -> Sigmoid   (decoder, reconstructs the input, scaled to [0,1])
```

Loss = MSE reconstruction + an L1 penalty on the first hidden layer's activations (encourages a sparse intermediate representation). See models/autoencoder.py (_AENet / MLPAutoencoder). Same two-stage pipeline as CNC-DEC/X-DEC: pretrain the autoencoder on reconstruction, then jointly fine-tune the encoder + a DEC clustering head (models/dec.py) so the latent space becomes clusterable — see Section 3.1 for the DEC algorithm.

4.2 How to run

```
py run_dec.py --neurons_e 8 --n_clusters 2
```

Or work interactively in DEC Runner.ipynb, which trains and then plots the same 3-panel results as CNC-DEC/X-DEC (PCA scatter, cluster-vs-label crosstab heatmap, cluster-assignment confidence histogram).

4.3 What can be changed

Flag	Meaning
--neurons_e	Embedding (latent) layer size (default 8) — the equivalent of CNC-VAE's --ls. In the notebook: MLPAutoencoder(n_features=X.shape[1], neurons_e=4).
--neurons_h	Hidden layer size (default 64).
--epochs / --bs	Autoencoder pretraining epochs (default 500) / batch size (default 64).
--n_clusters	Number of clusters to identify (default 2).
--dec_maxiter / --dec_update_interval / --dec_batch_size / --dec_tol	DEC self-training loop controls.
--n_init	K-means restarts for initializing cluster centers.
--clinical_mode	'curated' (default) or 'full'.
--label_col	Phenotype column reported against discovered clusters (default mgs_level).
--save_model	Save the trained autoencoder (weights .pt + a sidecar .architecture.csv).

Output directory name encodes --neurons_h/--neurons_e/--n_clusters: results/DEC/dec_H{neurons_h}_E{neurons_e}_k{n_clusters}/.

4.4 Packages needed

```
pip install -r requirements.txt
```

Installs: torch>=2.0, numpy>=1.24, pandas>=1.5, scikit-learn>=1.2, scipy>=1.10, matplotlib>=3.7.

4.5 Outputs

- {label_col}.npz — z (latent embedding), y_pred (cluster labels), y_proba, centroids, plus y/sample_id/classes.
- encoder_dec.pt (with --save_model).

Console output reports acc/ari/nmi against --label_col, both periodically and at the end.

4.6 Reloading a saved autoencoder

Unlike CNC-VAE/CNC-DEC/X-DEC (which bundle weights + architecture into one .pt file), this autoencoder's save_encoder() writes weights to path and the architecture hyperparameters to a separate sidecar CSV (path + '.architecture.csv'). Use models.autoencoder.load_autoencoder(path) to reload both together. As with the other variants, the latent size (neurons_e) is baked into the saved weights — you cannot change it without retraining from scratch.

5. X-DEC — X-shaped VAE + Deep Embedded Clustering

The dual-branch variant: instead of concatenating clinical + gene-expression features into one vector upfront (like CNC-VAE/CNC-DEC), X-DEC keeps them as two separate inputs that each get their own encoder branch, matched to their own data type and reconstruction loss, before merging into a shared bottleneck. Port of the DAM-IC DEC repo's xvae.py / DECLayer.py / clustering_functions.py (mixed numerical + binary data types path), rewritten in PyTorch.

- SET 1 (s1) must always be numerical — here, RNA-seq CPM expression — reconstructed with MSE.
- SET 2 (s2) must always be binary/one-hot — here, clinical/genotype dummy columns — reconstructed with binary cross-entropy.

5.1 How it works

```
 s1 (numeric: RNA-seq)        s2 (binary: clinical dummies)
        |                             |
        v                             v
 Linear->BatchNorm->act           Linear->BatchNorm->act    (separate branches, widths --ds1
/ --ds2)
        |                             |
        +--------------+---------------+
                       v
             concat -> Linear->BatchNorm->act    (shared bottleneck, width --ds12
                                                    -- branches merge here)
                       |
                +------+------+
                v             v
             fc_mu       fc_logvar             (-> latent z, dimension --ls)
                       |
                       v
        shared decoder trunk -> splits back into s1-branch / s2-branch
                |                   |
                v                   v
         s1 reconstruction    s2 reconstruction (logits -> sigmoid)
          (MSE loss)           (BCE-with-logits loss)
```

Model code: models/xvae.py (_XVAENet/xvae class), math helpers in models/common.py (KL/MMD/reparameterization — same as CNC-VAE's, shared across a dual-branch architecture). Same two-stage DEC pipeline as CNC-DEC/DEC: see models/dec.py's XDEC class (same Student's-t soft-assignment / target-sharpening algorithm as the other two variants — see Section 3.1 — just calling xvae.net.encode(x1, x2) instead of a single-input encode(x)).

5.2 How to run

```
py run_xdec.py --ds1 48 --ls 16 --n_clusters 2
```

Or work interactively in XDEC Runner.ipynb, which trains, saves results + an encoder checkpoint to results/manual_run/, then plots the same 3-panel results as CNC-DEC/DEC (added so all three DEC variants are visually comparable): PCA scatter, cluster-vs-label crosstab heatmap, cluster-assignment confidence histogram.

5.3 What can be changed

Flag	Meaning
--ls	Latent dimension size (default 16) — the main knob to experiment with.
--ds1	Hidden width of the numeric (RNA) branch (default 48).
--ds2	Hidden width of the binary (clinical) branch (defaults to --ds1).
--ds12	Hidden width of the shared bottleneck where both branches merge (defaults to --ds1).
--n_clusters	Number of clusters to identify (default 2).
--distance / --beta	VAE regularizer (kl or mmd) and its weight.
--unweighted	By default, the two branches' reconstruction losses are pooled into one shared per-feature average (a branch with more features gets more weight). Pass this flag to instead average each branch independently first.
--epochs / --bs	X-VAE pretraining epochs (default 150) / batch size (default 32).
--dec_maxiter / --dec_update_interval / --dec_batch_size / --dec_tol	DEC self-training loop controls.
--n_init	K-means restarts for initializing cluster centers.
--clinical_mode	'curated' (default) or 'full'.
--label_col	Phenotype column reported against discovered clusters (default mgs_level).
--save_model	Save the trained encoder as encoder_xvae.pt (for the SHAP workflow — see XVAEEncoderMu).
--stability	Repeated K-fold cluster-stability check (slow).

A one-time warning prints if the clinical branch has more columns than there are samples (--clinical_mode full on this cohort size) — a nudge toward --clinical_mode curated rather than a hard failure. Output directory encodes --ls/--ds1/--ds2/--ds12/--distance/--beta/--n_clusters: results/XDEC_Clin+mRNA_integration/xdec_LS_{ls}_DS1_{ds1}_DS2_{ds2}_DS12_{ds12}_{distance}_beta_{beta}_k{n_clusters}/.

5.4 Packages needed

```
pip install -r requirements.txt
```

Installs: torch>=2.0, numpy>=1.24, pandas>=1.5, scikit-learn>=1.2, scipy>=1.10, matplotlib>=3.7 (needed for XDEC Runner.ipynb's plots).

5.5 Outputs

- {label_col}.npz — z, y_pred, y_proba, centroids, plus y/sample_id/classes.
- xdec_latent_embedding.csv — same embedding, drop-in shape for the XGBoost pipeline.
- encoder_xvae.pt (with --save_model); cluster_stability.csv (with --stability).

Console output reports acc/ari/nmi against --label_col.

5.6 Reloading a saved encoder

encoder_xvae.pt bundles weights + architecture (s1_input_size, s2_input_size, ds1, ds2, ds12, ls, dropout, act) into one file — reload with models.xvae.load_xvae_model(path). As with the other variants, the latent size is fixed at training time; there's no way to change --ls on a saved checkpoint without retraining. For tools that expect a single-tensor model input (e.g. SHAP), use XVAEEncoderMu (in models/xvae.py) — it wraps the two-branch encoder to accept one concatenated tensor [s1 | s2] and splits it internally.

6. MEGENA_AMI — Gene Co-expression Network Analysis (R)

Builds a gene co-expression network from the AMD RNA-seq cohort using MEGENA (Multiscale Embedded Gene Co-Expression Network Analysis), then extracts per-module "eigengenes" and tests whether they differ between MGS grades. "AMI" = Adjusted Mutual Information — the gene-gene similarity measure used here to build the network (computed upstream in Scripts/Python Scripts/AMI.ipynb), as opposed to Pearson correlation, JSD, or Wasserstein distance used in the sibling MEGENA_BD / MEGENA_SKL_JSD / MEGENA_WD / MEGENA_WGCNA folders (not covered here). This is a completely separate R workflow from the Python codebases in Sections 2–5 — no shared code, just the same underlying cohort data.

Before running anything: all three scripts use hardcoded absolute paths (C:/Users/Brayan Gutierrez/Desktop/RNAseq-AMD/...). On a new machine, find-and-replace that path prefix with wherever this repo lives locally — the scripts will error immediately otherwise (file not found).

6.1 The 3 scripts in this folder

Script	Cohort / distance file	What it adds
MEGENA_AMI_1_100.R	MGS1 only (mild); Dataset/AMI/ami_edges_control.csv	Builds a network + modules for MGS1 alone, reports per-module eigengene summary stats.
MEGENA_AMI_4_100.R	MGS4 only (severe); Dataset/AMI/ami_edges_late.csv	Identical to the above, for MGS4.
MEGENA_AMI_1_4_100.R	MGS1+MGS4 combined; Dataset/AMI/ami_edges.csv	Builds one network on the combined cohort, plus tests each module's eigengene for a difference between MGS1 vs. MGS4 (Wilcoxon rank-sum + t-test, BH-corrected).

The _1_100 / _4_100 pair differ from each other in exactly 3 lines (input file, plot title, mgs_level filter) — diffing them is the fastest way to see what changes between a single-group run and its counterpart.

6.2 How it works (pipeline, same shape in all 3 scripts)

```
# 1. Load the precomputed AMI gene-gene similarity edges + expression + gene metadata
distance = read.csv(".../Dataset/AMI/ami_edges*.csv")   # columns: from, to, weight
genes    = read.csv(".../Dataset/aak100_cpmdat.csv")     # samples x genes CPM matrix
info     = read.delim(".../Dataset/gene_info.tsv")       # ensembl_gene_id -> gene symbol
etc.

# 2. Build a Planar Filtered Network (PFN) - MEGENA's network-sparsification step
pfn = calculate.PFN(distance)
pfn_g = igraph::graph_from_data_frame(d = pfn, directed = FALSE)

# 3. Run MEGENA's multiscale clustering to find modules (+ hub genes) on that network
meg = do.MEGENA(g = pfn_g, mod.pval = 0.05, hub.pval = 0.05, min.size = 10, n.perm = 100)
module_summary = MEGENA.ModuleSummary(meg)
modules = module_summary$modules   # named list: module id -> vector of gene ids

# 4. Visualize interactively (nodes colored/grouped by module,
```

```
#    hover = gene + module + degree)
visNetwork(nodes, edges, main = "...") %>% visIgraphLayout() %>% visOptions(...) %>%
visLegend()

# 5. Score each module: modularity, conductance, density, transitivity,
#    avg. intra-module correlation

# 6. Compute one "eigengene" per module = PC1 of that module's genes' expression (prcomp)
#    -> a single per-sample number summarizing that module's expression level

# 7. (1_4_100 only) Test each module eigengene for a difference between MGS1 vs MGS4
```

6.3 What can be changed

All of these are plain variable edits at the top of the script — no command-line flags. These are interactive R scripts, run top-to-bottom in RStudio.

What	Where / notes
Input distance file	distance = read.csv(...) — swap in a different Dataset/AMI/ami_edges*.csv variant (there are several precomputed: _QNclass, _QNglobal, _QS, etc.) to compare normalization choices.
mod.pval / hub.pval	do.MEGENA(...) call — significance thresholds for calling something a module / a hub gene. Lower = stricter (fewer, more confident modules/hubs).
min.size	do.MEGENA(...) call — minimum genes per module; smaller modules are dropped.
n.perm	do.MEGENA(...) call — number of permutations for the significance tests above; higher = more stable p-values, slower.
Which MGS grade	In _1_100/_4_100: subset(genes, mgs_level == "MGS1") — change to any other grade present in mgs_level.
Paired vs. unpaired testing	In _1_4_100.R, the commented-out pair_col block (~line 213) — by default there's no donor/pair ID so the script falls back to an unpaired Wilcoxon test. Uncomment + set pair_col to enable a paired signed-rank test if you have one.

6.4 Packages needed

```
install.packages(c(
  "MEGENA",        # co-expression network module detection - see note below
  "igraph",        # graph object + graph metrics (modularity, conductance, density,
transitivity)
  "visNetwork",     # interactive network visualization
  "RColorBrewer",   # color palettes for the plots
  "readr",          # fast CSV/TSV reading
  "dplyr",          # data wrangling (group_by/summarise, used in the paired-test path)
  "ggplot2",        # plotting (only used in the commented-out extension blocks)
  "pheatmap",       # heatmap plotting (only used in the commented-out extension blocks)
  "reshape2",       # long-format reshaping (only used in the commented-out extension
blocks)
  "tidyr"           # pivot_wider, used via tidyr:: in the paired-test path
))
```

MEGENA is on CRAN (install.packages("MEGENA") should just work). If that ever fails, the development version is on GitHub:

```
install.packages("devtools")
```

```
devtools::install_github("songw01/MEGENA")
```

Source: MEGENA on CRAN/METACRAN (r-pkg.org/pkg/MEGENA); songw01/MEGENA on GitHub.

6.5 Data files needed

- Dataset/aak100_cpmdat.csv — RNA-seq CPM expression matrix, samples x genes, with an mgs_level phenotype column (MGS1/MGS4/...).
- Dataset/gene_info.tsv — Ensembl gene ID -> gene symbol / genomic location lookup, used to label network nodes.
- Dataset/AMI/ami_edges.csv, ami_edges_control.csv, ami_edges_late.csv — precomputed gene-gene AMI similarity edge lists (from, to, weight), one per cohort variant. Generated upstream by ../Python Scripts/AMI.ipynb — if you need to regenerate them (new samples, different gene set, etc.), that notebook is the place to start; it's a separate Python workflow, not part of this R folder.

6.6 How to run

Open the .Rproj in Scripts/ (or just open the .R file in RStudio), update the hardcoded paths at the top to match your machine, then source top-to-bottom — either the whole file (Source) or section-by-section if you want to inspect intermediate objects (pfn_g, meg, module_summary, ME_df) along the way:

```
# from an R console, with the working directory set anywhere (paths are absolute):
source("MEGENA_AMI_1_4_100.R")   # combined MGS1+MGS4 cohort + differential eigengene test
source("MEGENA_AMI_1_100.R")     # MGS1-only network
source("MEGENA_AMI_4_100.R")     # MGS4-only network
```

Each script is meant to be run interactively, not headlessly — the visNetwork(...) call opens an interactive HTML widget in the RStudio Viewer pane, and several print(...) calls are how you're meant to inspect results, not values that get saved to disk automatically.

6.7 Outputs (in-session objects, not auto-saved to disk)

Object	What it is
pfn_g	The igraph network object (Planar Filtered Network).
meg / module_summary	Raw and summarized MEGENA output; module_summary$modules is the named list of gene modules.
module_metrics	Data frame: per-module modularity/conductance/density/transitivity/avg. correlation.
ME_df	Per-sample eigengene matrix (one column per module, PC1 of that module's genes) + Phenotype.
module_results	(1_4_100 only) Per-module Wilcoxon + t-test results vs. MGS1/MGS4, with BH-adjusted FDR — the table to look at for “which modules differ between mild and severe AMD.”

If you want these saved to disk rather than just left in the R session, add e.g. write.csv(module_results, "module_results.csv") — none of the three scripts currently do this by default.

Appendix — Package Install Quick Reference

Python (pip)

Project	Command (run from inside that project's folder)
CNC-VAE	pip install -r requirements.txt
CNC-DEC	pip install -r requirements.txt
DEC	pip install -r requirements.txt
X-DEC	pip install -r requirements.txt

All four requirements.txt files pin similar core packages (torch, numpy, pandas, scikit-learn); CNC-VAE additionally needs shap, and all three DEC variants need scipy (for the Hungarian-matching clustering-accuracy metric) and matplotlib (for their *Runner.ipynb notebooks).

R

```
install.packages(c(
  "MEGENA", "igraph", "visNetwork", "RColorBrewer", "readr",
  "dplyr", "ggplot2", "pheatmap", "reshape2", "tidyr"
))
```

Only needed for Section 6 (MEGENA_AMI) — completely independent of the Python setup above.